

M I M A N U A L D E

PYTHON

Apuntes de clase — Programa de formación Python

B L O Q U E S 0 - 8

Abril 2026

R E S U M E N D E P A T R O N E S

Parte 1

- 01 — Condición encadenada
- 02 — for con range()
- 03 — Recorrer dos listas
- 04 — Acumulador
- 05 — Contador condicional
- 06 — Campeón vigente
- 07 — Bucle infinito controlado

Parte 2

- 08 — enumerate()
- 09 — Desempaquetado de tuplas
- 10 — Evitar el -1 en menús
- 11 — Lista de tuplas
- 12 — Diccionario vs tuplas
- 13 — .keys() .values() .items()
- 14 — Menús y submenús
- 15 — Despacho por diccionario
- 16 — Lista de diccionarios

[python.org](#) — [docs.python.org/es](#)
Python para todos — Raúl González Duque
La biblia de Python — QA sin filtros

C O N T E N I D O

0	Bloque 0 — Lógica de programación <i>Algoritmos · Pseudocódigo · Árbol de descomposición · Lógica Booleana</i>
1	Bloque 1 — Fundamentos absolutos <i>Variables · Operadores · Strings · f-strings · Format Specifiers</i>
2	Bloque 2 — Control de flujo <i>Condicionales · Bucles · break/continue · Patrones 01–07 · Mapas de proceso</i>
3	Bloque 3 — Estructuras de datos <i>Listas · Diccionarios · Tuplas · Sets · Patrones 08–16</i>
4	Bloque 4 — Funciones <i>def · parámetros · scope · return · lambda · map() · filter()</i>
5	Bloque 5 — Organización del código <i>Módulos · venv/pip · Excepciones · Archivos de texto · datetime</i>
6	Bloque 6 — OOP Básica <i>Clases · Objetos · __init__ · Métodos · Herencia</i>
7	Bloque 7 — Archivos, JSON y Bases de Datos <i>open · read/write · json · csv · SQLite · sqlite3</i>
8	Bloque 8 — Librerías y automatización <i>Rich · Pandas · OpenPyXL · NumPy · Matplotlib · Seaborn · Plotly · Streamlit · SciPy · Polars · urwid · Dask</i>

BLOQUE 0

LÓGICA DE PROGRAMACIÓN

Pensamiento Algorítmico · Pseudocódigo · Lógica Booleana

Pensamiento Algorítmico

¿Qué es un algoritmo?

Secuencia de pasos ordenados, finitos y sin ambigüedad que resuelven un problema.

ENTRADA	PROCESO	SALIDA
¿Qué datos necesitas?	¿Qué pasos realizas?	¿Qué resultado entregas?

Regla de oro: Verificar el algoritmo con datos reales ANTES de escribir código. Ese hábito separa a quien programa bien de quien prueba a ver qué sale.

Práctica — Algoritmo de Planeación

```
ENTRADA
cantidad_carpetas, cantidad_analistas, cantidad_supervisores
fecha_inicio, fecha_final
meta_diaria_analistas, meta_diaria_supervisores

PROCESO
dias = fecha_final - fecha_inicio
TABLA 1:
    meta_grupal_analistas = cantidad_analistas * meta_diaria_analistas
    meta_grupal_supervisores= cantidad_supervisores*
    meta_diaria_supervisores

    TABLA 2 (iterar por cada día N de 1 hasta dias):
        meta_ind_analista = meta_diaria_analistas * N
        meta_ind_supervisor= meta_diaria_supervisores * N
        meta_grp_analistas = meta_ind_analista * cantidad_analistas
        meta_grp_supervisores= meta_ind_supervisor* cantidad_supervisores
        avance = meta_grp_supervisores / cantidad_carpetas * 100

SALIDA
TABLA 1: ROL | Cantidad | Meta Individual/Dia | Meta Grupal/Dia
```

BLOQUE 8

AUTOMATIZACIÓN Y DATOS

pandas · matplotlib · plotly · Rich · OpenPyXL · NumPy · Polars · Dask · Streamlit · Seaborn · SciPy · urwid

- Categoría 1 — Terminal y UI
 - Rich
 - Urwid
- Categoría 2 — Datos Tabulares
 - Pandas
 - NumPy
 - Polars
 - Dask
- Categoría 3 — Manejo de Excel
 - OpenPyXL
- Categoría 4 — Visualización de Datos
 - Matplotlib
 - Seaborn
 - Plotly
- Categoría 5 — Aplicaciones Web con Streamlit
 - Streamlit
 - SciPy
-

BLOQUE 7

ARCHIVOS, JSON Y BASES DE DATOS

open · read/write · json

Contenido en desarrollo. Se completará próxima clase.

- CSV — Datos Tabulares en Texto Plano
- JSON — Intercambio de Datos
- SQLite — Base de Datos Relacional Local

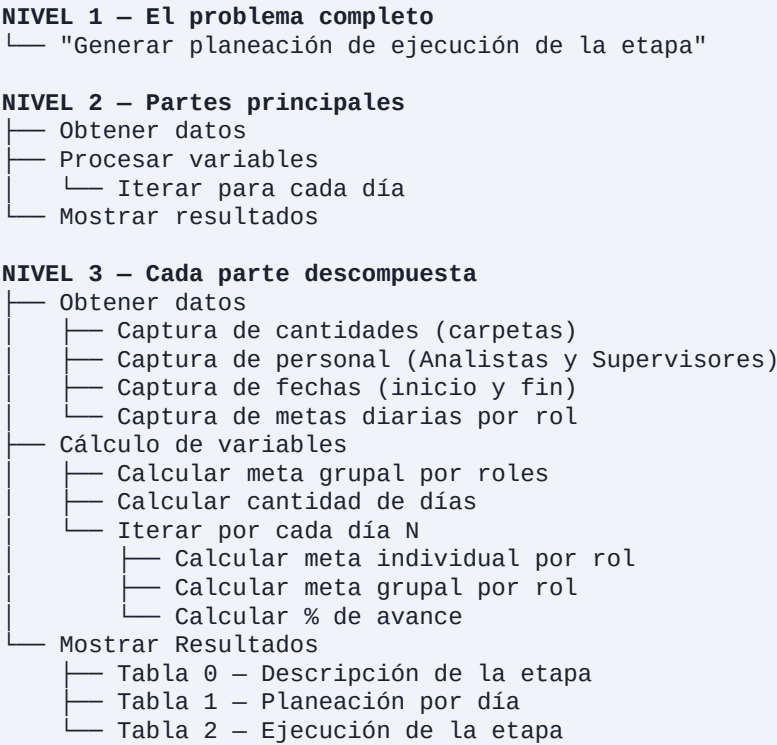
TABLA 2: FECHA		META IND. A		META IND. S		META GRP A		META GRP S		%
----------------	--	-------------	--	-------------	--	------------	--	------------	--	---

Descomposición de Problemas

Tomar un problema grande y dividirlo en partes más pequeñas que se pueden resolver una por una. Regla de una responsabilidad: cada bloque debe hacer UNA sola cosa y hacerla bien.

Regla de oro: Si describes lo que hace un bloque y usas la palabra "y", probablemente necesitas partirlo.

Diagrama de Árbol — Planeación



Pseudocódigo

Usa palabras, NO sintaxis de ningún lenguaje. La indentación importa: lo que está adentro pertenece a la estructura de arriba.

Pseudocódigo	Python	Pseudocódigo	Python
SI	if	SALIR	break
SINO SI	elif	CONTINUAR	continue
SINO	else	FUNCIÓN	def
PARA	for	Y	and
MIENTRAS	while	O	or
RETORNAR	return	NO	not

Diagramas de Flujo

Símbolo	Representa
Ovalo	Inicio / Fin del programa
Rectángulo	Proceso (operación, cálculo)
Rombo	Decisión (SI / NO)
Paralelogramo	Entrada o Salida de datos
Flechas	Flujo / dirección de ejecución

Trazabilidad Paso a Paso

Seguir la ejecución de un programa línea por línea con valores reales, como si fueras Python. Útil para detectar **bugs** antes de ejecutar y verificar la lógica en papel.

Cómo se hace: Tabla de Trazabilidad

Cada fila = una vuelta del bucle. Cada columna = una variable clave.

```
# Variables de ejemplo
```

BLOQUE 6

OOP BÁSICA

Clases · Objetos · Atributos · Métodos

Contenido en desarrollo. Se completará próxima clase.

- Definir una clase con class
- __init__ — constructor e inicialización
- Atributos de instancia y de clase
- Métodos — comportamiento de los objetos
- Herencia — reutilizar y extender clases

BLOQUE 5

ORGANIZACIÓN DEL CÓDIGO

Módulos · Manejo de errores · Archivos

Contenido en desarrollo. Se completará próxima clase.

- Módulos e Importaciones
- Módulos esenciales de la biblioteca estándar (os, math, random, sys, pathlib, collections, itertools)
- Entornos Virtuales y Gestión de Paquetes
- Manejo de Excepciones try / except / else /finally — manejo de errores
- Tipos de excepciones comunes
- Lectura y escritura de archivos de texto
- Fechas y Tiempo — datetime

```
meta_diaria_supervisores = 24
cantidad_supervisores    = 3
cantidad_carpetas        = 100

for i in range(0, 4):
    meta_ind = meta_diaria_supervisores * i
    meta_grp = meta_ind * cantidad_supervisores
    if meta_grp >= cantidad_carpetas:
        meta_grp = cantidad_carpetas
```

i	meta_ind	meta_grp	¿Límite?
0	0	0	No
1	24	72	No
2	48	144 → 100	Sí
3	72	216 → 100	Sí

Regla de oro: Si en la tabla ves el mismo valor en "¿Límite?" más de una vez, hay una inconsistencia que corregir.

Lógica Booleana

True / False

Tipo de dato que solo tiene dos valores posibles:

- Se comportan como **True:** 1, -1, "hola", [1,2], {"a":1}
- Se comportan como **False:** 0, 0.0, "", [], {}, None

Operador and — exigente

Devuelve True SOLO SI todas las condiciones son verdaderas.

```
# Solo descuento si ES frecuente Y compra >= 1000
if es_frecuente and venta >= 1000:
    aplicar_descuento()
```

Condición 1	Condición 2	and
True	True	True ✓
True	False	False ✗
False	True	False ✗
False	False	False ✗

Operador or — generoso

Devuelve True si AL MENOS UNA condición es verdadera.

```
# Alerta si analista o supervisor incumplió
if analista_incumple or supervisor_incumple:
    enviar_alerta()
```

Condición 1	Condición 2	or
True	True	True ✓
True	False	True ✓
False	True	True ✓
False	False	False ✗

Operador not — inversor

Condición	not
True	False ✗
False	True ✓

Operadores de Comparación

Operador	Significado	Ejemplo
==	Igual que	x == 5

BLOQUE 4

FUNCIONES

def · parámetros · scope · return

Contenido en desarrollo. Se completará próxima clase.

- Definir y llamar funciones — def nombre(param):
- Parámetros por defecto y keyword arguments
- Return — retornar uno o múltiples valores
- Scope local vs global
- Funciones lambda — funciones anónimas
- Funciones de orden superior — map(), filter()


```
etapas_aprobadas = {"Física", "Entrevista", "Física"}
# {"Física", "Entrevista"} <- elimina duplicados

A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
A | B    # Unión:      {1, 2, 3, 4, 5, 6}
A & B    # Intersección: {3, 4}
A - B    # Diferencia:  {1, 2}
```

 Ejercicio: Diccionario de convocatorias

Crea un diccionario "convocatorias" con las claves EAAB-01, EAAB-02, EAAB-03.
Cada convocatoria tiene: cargo, vacantes, inscritos.
Calcula la ratio inscritos/vacantes para cada una usando .items().
Usa dict comprehension para filtrar solo las que tienen más de 10 inscritos por vacante.
Muestra los resultados formateados: f'{cargo:<20} {ratio:.1f} inscritos/vacante'

Operador	Significado	Ejemplo
!=	Diferente de	x != 0
>	Mayor que	nota > 70
<	Menor que	edad < 18
>=	Mayor o igual que	puntaje >= 80
<=	Menor o igual que	nivel <= 3

== vs is

Operador	Pregunta que hace	Cuando usarlo
==	¿Los valores son iguales?	El 99% del tiempo
is	¿Es el mismo objeto en memoria?	Solo con None, True, False

B L O Q U E 1

FUNDAMENTOS ABSOLUTOS

Variables · Operadores · Strings · f-strings · Format Specifiers

Variables y Tipos de Datos

Espacio en memoria donde se guardan y recuperan los datos. El nombre no puede coincidir con palabras reservadas ni tener espacios.

Tipo	Representación	Descripción	Ejemplo
Entero	int	Números sin decimales	edad = 25
Decimal	float	Números con decimales	precio = 9.99
Complejo	complex	Parte real e imaginaria	2 + 3j
Texto	str	Cadena entre comillas	"hola"
Booleano	bool	Verdadero o falso	True / False

Regla de oro: Las variables distinguen mayúsculas: "Nombre" y "nombre" son variables distintas y pueden almacenar datos diferentes.

Palabras Reservadas

Identificadores de uso exclusivo de Python. NO se pueden usar como nombres de variables, métodos u objetos.

Palabras Reservadas							
and	As	assert	break	class	continue	def	del
elif	else	except	finally	for	from	global	if
import	in	is	lambda	nonlocal	not	or	pass
raise	return	try	while	with	yield	True	False

```
# Filtrar
validos = [r for r in registros if r["estado"] == True]

# Exportar a Excel
import pandas as pd
df = pd.DataFrame(validos)
df.to_excel("registros_validos.xlsx", index=False)
```

Tuplas

Colección ORDENADA e INMUTABLE. Igual que una lista pero no puede modificarse después de creada.

```
coordenadas = (4.7110, -74.0721) # Bogotá
rgb          = (255, 128, 0)

coordenadas[0] # 4.7110

# Desempaquetado
lat, lon = coordenadas
```

Regla de oro: Usa tupla cuando los datos NO deben cambiar: coordenadas, fechas fijas, constantes de configuración.

◆ Patrón 09 — Desempaquetado de tuplas [TUPLAS]

Cuando una tupla tiene 2+ valores conocidos, siempre desempaqueta — el código queda más legible y sin índices manuales.

Por índice (verbose):
tasas[0][0] # "USD"

Desempaquetado directo:
moneda, tasa = tasas[0]

Desempaquetado en el for:
for i, (moneda, tasa) in tasas.items():
 print(f"{i}. {moneda}: {tasa:,}")

Sets (Conjuntos)

Colección DESORDENADA de elementos Únicos. No permite duplicados. Útil para operaciones de conjuntos.

(igual que `.keys()`).

```
tasas = {1: ("USD", 4000), 2: ("EUR", 4300)}

tasas.keys()      # 1, 2
tasas.values()    # ("USD", 4000), ("EUR", 4300)
tasas.items()     # (1, ("USD", 4000)), (2, ("EUR", 4300))

for i, (moneda, tasa) in tasas.items():
    print(f"{i}. {moneda} (Tasa: {tasa:,})")
```

◆ Patrón 14 — Estructura de menús y submenús *[DICCIONARIO + TUPLAS]*

Diccionario para navegar (el usuario elige por número), tupla para transportar (carga texto y el siguiente nivel).

```
menu = {
    1: ("Calculadora de horas", {
        1: "Horas totales",
        2: "Horas diarias",
    }),
    2: ("Calculadora directa", None),
    3: ("Salir", None)
}
titulo, submenu = menu[1]    # sin -1
```

◆ Patrón 15 — Despacho por diccionario *[DICCIONARIO + FUNCIONES]*

Reemplaza cadenas largas de `elif` con un diccionario que mapea opciones a funciones. Agregar una opción = 1 línea.

```
acciones = {
    1: calculadora_de_tiempos,
    2: calculadora_de_horas,
    3: calculadora_de_semestres
}
acciones[opcion]()    # ejecuta la función elegida
```

◆ Patrón 16 — Lista de diccionarios para registros *[LISTAS + DICCIONARIOS]*

Es el puente natural hacia pandas y bases de datos. Lista de diccionarios = datos tabulares en Python.

```
registros = [
    {"institucion": "Universidad Libre", "cargo": "Dev", "estado": True},
    {"institucion": "Empresa X", "cargo": "Analista", "estado": False},
]
```

Operadores Aritméticos

Nombre	Operador	Ejemplo	Resultado
Suma	+	10 + 3	13
Resta	-	10 - 3	7
Multiplicación	*	10 * 3	30
División	/	10 / 3	3.333...
División entera	//	10 // 3	3
Módulo	%	10 % 3	1
Exponente	**	2 ** 8	256

Operadores de Asignación

Operador	Equivale a	Ejemplo
=	x = valor	x = 5
+=	x = x + y	x += 3
-=	x = x - y	x -= 2
*=	x = x * y	x *= 4
/=	x = x / y	x /= 2
//=	x = x // y	x //= 3
**=	x = x ** y	x **= 2
%=	x = x % y	x %= 5

Strings a Fondo

Un string es una cadena de caracteres: letras, números y símbolos entre comillas simples o dobles.

Slicing — Subcadenas

Sintaxis: `variable[inicio:fin:paso]`

```
texto = "Aprendiendo Python"
texto[0:11]      # "Aprendiendo"
texto[-6:]       # "Python"
texto[::2]       # cada 2 caracteres
texto[::-1]      # invertido
```

Concatenación — f-strings básicos

Los f-strings permiten insertar expresiones directamente dentro de una cadena de texto.

```
nombre = "Ana"
edad    = 28
cargo   = "Analista"

# Sintaxis: f"texto {variable} texto"
print(f"{nombre} tiene {edad} años y es {cargo}")
# Ana tiene 28 años y es Analista

# Expresiones directas dentro de {}
print(f"El doble de la edad: {edad * 2}") # 56
print(f"En mayúsculas: {nombre.upper()}") # ANA
```

F-Strings y Format Specifiers

Sintaxis completa: `f'{expresión:especificador}'` — el especificador controla cómo se muestra el valor.

Tabla Cheat Sheet — Los más usados

Objetivo	f-string	Resultado
Valor monetario COP	<code>f'\${valor:,.0f}'</code>	\$1,234,567

```
aprobados = {nombre: pts for nombre, pts in puntajes.items()
              if pts >= 80}
# {"Laura": 91, "Camila": 88}
```

- Formas:**
- 01. La parte antes del for define qué va al diccionario → siempre clave: valor
 - 02. La parte del for es igual a cualquier bucle normal
 - 03. El if (Condición) es opcional — solo lo pones si quieres filtrar

Resumen en una línea por forma:

- `{k: v for k, v in iterable}`
- `{k: v for k, v in iterable if condición}`

◆ **Patrón 10 — El -1 en menús y cómo evitarlo** *[ÍNDICES]*
Cuando el usuario elige por número en un menú, un diccionario con claves numéricas elimina el -1.

```
# El problema: lista indexa desde 0, usuario desde 1
moneda, tasa = lista[opcion - 1] # -1 necesario

# Solución: diccionario con claves 1, 2, 3...
tasas = {1: ("USD", 4000), 2: ("EUR", 4300), 3: ("BRL", 800)}
moneda, tasa = tasas[opcion] # sin -1
```

◆ **Patrón 12 — Diccionario vs lista de tuplas** *[DICcionario]*
¿Buscas por clave? → diccionario. ¿Solo iteras? → lista de tuplas.

¿Buscas por clave?	Estructura recomendada
Menú: usuario elige por número/código	Diccionario <code>menu[opcion]</code>
Solo recorrer todos en orden	Lista de tuplas
Múltiples registros del mismo tipo	Lista de diccionarios
Objeto con atributos nombrados	Diccionario <code>obj["campo"]</code>

◆ **Patrón 13 — .keys() .values() .items()** *[DICcionario]*
Usa .items() cuando necesitas clave y valor juntos — es el más común. for k in dic itera solo las claves

Método	Qué hace	Ejemplo
values()	Devuelve todos los valores	for v in dic.values()
items()	Devuelve tuplas (clave, valor)	for k,v in dic.items()
pop(k)	Elimina la clave k y devuelve su valor	dic.pop("nombre")
update(d2)	Inserta o actualiza con otro diccionario	dic.update({"x":1})
clear()	Vacía el diccionario (queda {})	dic.clear()
del dic[k]	Elimina el par clave-valor directamente	del dic["cargo"]

Regla de oro: Usa `[]` cuando estás seguro de que la clave existe. Usa `.get()` cuando los datos vienen del exterior (Excel, usuario, BD) y la clave podría faltar.

Dict Comprehension

Una comprensión es una forma de construir una estructura de datos en una sola expresión, en lugar de crearla vacía y llenarla con un bucle.

Siempre hay una versión "larga" equivalente. La comprensión no hace nada nuevo — solo es más compacta.

Regla de oro: Si al leerla en voz alta no se entiende en 5 segundos, escríbela como bucle. La comprensión existe para claridad, no para impresionar.

```
{clave: valor for clave, valor in iterable}
{clave: valor for clave, valor in iterable if condicion}
```

```
# Ejemplo: solo aspirantes aprobados
puntajes = {"Carlos":72, "Laura":91, "Andres":65, "Camila":88}
```

Objetivo	f-string	Resultado
Porcentaje legible	<code>f'{ratio:.1%}'</code>	75.3%
Decimal 2 cifras	<code>f'{numero:.2f}'</code>	3.14
Sin decimales	<code>f'{numero:.0f}'</code>	4
Código con ceros	<code>f'PROD-{id:06d}'</code>	PROD-000042
Alinear izquierda (15)	<code>f'{texto:<15} '</code>	hola
Alinear derecha (15)	<code>f'{numero:>15} '</code>	1234
Centrado (15)	<code>f'{texto:^15} '</code>	hola
Relleno con *	<code>f'{texto:*^15} '</code>	*****hola*****
Fecha colombiana	<code>f'{dt:%d/%m/%Y}'</code>	15/03/2026
Debug (Python 3.8+)	<code>f'{var = }'</code>	var = 42

Ejemplo Práctico — Tabla de reporte

```
productos = [
    ("Laptop Dell",      2, 3500000),
    ("Monitor LG",       2,  850000),
    ("Teclado",          4,  120000),
]

cabecera = f'{"PRODUCTO":<20} {"CANT":>4} {"PRECIO":>12} {"SUBTOTAL":>12}'
print(cabecera)
print("=" * 50)
total = 0
for nombre, cant, precio in productos:
    sub = cant * precio
    total += sub
    print(f'{nombre:<20} {cant:>4} ${precio:>10,.0f} ${sub:>10,.0f}')
print("=" * 50)
pie = f'{"TOTAL A PAGAR":<30} ${total:>10,.0f}'
print(pie)
```

Regla de oro: Usa `:.0f` para cualquier valor monetario en Colombia (separador de miles, sin

decimales). Usa `:.1%` para porcentajes legibles. Usa `f{var = }` durante el desarrollo para depurar rápido.

Funciones Integradas (Built-ins)

Función	Qué hace	Sintaxis
<code>len()</code>	Longitud de una cadena o colección	<code>len("hola")</code>
<code>print()</code>	Muestra información en pantalla	<code>print("texto")</code>
<code>input()</code>	Solicita dato al usuario (siempre <code>str</code>)	<code>input("Nombre: ")</code>
<code>type()</code>	Tipo de dato de una variable	<code>type(x)</code>
<code>int()</code>	Convierte a entero	<code>int("25")</code>
<code>float()</code>	Convierte a decimal	<code>float("3.14")</code>
<code>str()</code>	Convierte a texto	<code>str(100)</code>
<code>bool()</code>	Convierte a booleano	<code>bool(0)</code>
<code>max()</code>	Valor máximo de un iterable	<code>max(lista)</code>
<code>min()</code>	Valor mínimo de un iterable	<code>min(lista)</code>
<code>sum()</code>	Suma todos los elementos	<code>sum(lista)</code>
<code>range()</code>	Genera secuencia de números	<code>range(1, 10, 2)</code>
<code>enumerate()</code>	Recorre con índice y valor	<code>enumerate(lista)</code>
<code>zip()</code>	Une dos listas elemento a elemento	<code>zip(lista1, lista2)</code>
<code>help()</code>	Documentación de un objeto	<code>help(str)</code>

Creación: Se realiza al indicar entre llaves `{}`, y se señala cada par como "clave": valor separados por dos puntos `“:”`, los pares que se incluirán en el diccionario se separan con `“,”`:

Operaciones Directas []:

Ejemplo:

```
estudiante = {"nombre": "Aria", "edad": 12}
```

- Crear, Añadir y actualizar un nuevo par clave-valor:** Si la clave no existe, se añade al diccionario, Si la clave ya existe, se sobrescribe el valor anterior

```
estudiante["calificación"] = "7°"
```

- Acceder a un valor:**

```
nombre = estudiante["nombre"] # Devuelve 'Aria'
```

```
convocatorias = {
    "EAAB-01": {"cargo": "Ingeniero", "vacantes": 20},
    "EAAB-02": {"cargo": "Técnico", "vacantes": 50},
}

convocatorias["EAAB-01"]["cargo"] # "Ingeniero"
convocatorias["EAAB-03"] = {"cargo": "Auxiliar", "vacantes": 80} # agrega
```

Métodos de Diccionarios

Método	Qué hace	Ejemplo
<code>get(k, d)</code>	Busca clave <code>k</code> ; retorna <code>d</code> si no existe	<code>dic.get("cargo", "N/D")</code>
<code>keys()</code>	Devuelve todas las claves	<code>for k in dic.keys()</code>

◆ Patrón 03 — Recorrer dos listas a la vez [FOR + LISTAS]

i es la posición, no un dato. Con esa posición entras a cualquier lista al mismo tiempo.

```
nombres = ["Ana", "Luis", "Sofia"]
notas    = [50, 78, 80]

for i in range(len(nombres)):
    print(nombres[i], notas[i])

# Mas Pythonico con zip():
for nombre, nota in zip(nombres, notas):
    print(nombre, nota)
```

◆ Patrón 08 — range(len()) vs enumerate() [FOR + ENUMERATE]

Si usas range(len(...)) para recorrer una lista, casi siempre hay una alternativa más limpia con enumerate().

```
# Verbose:
for i in range(len(monedas)):
    print(f"{i+1}. {monedas[i]}")

# Pythonico:
for i, moneda in enumerate(monedas, start=1):
    print(f"{i}. {moneda}")
```

◆ Patrón 11 — Listas paralelas vs lista de tuplas [LISTAS / TUPLAS]

Si dos listas siempre van juntas, úsalas como lista de tuplas desde el inicio.

Listas paralelas	Lista de tuplas
monedas = ["USD", "EUR"]	tasas = [("USD",4000), ("EUR",4300)]
tasas = [4000, 4300]	— datos juntos desde el inicio —
Necesita zip para unir las	Modificar = tocar 1 solo lugar

Diccionarios

Colecciones que relacionan una CLAVE con un VALOR. Las claves son únicas e inmutables.

Métodos de Strings

Método	Qué hace	Ejemplo
strip()	Elimina espacios al inicio y fin	" hola ".strip()
rstrip()	Elimina al inicio (izquierda)	" hola".rstrip()
lstrip()	Elimina al final (derecha)	"hola ".lstrip()
upper()	Todo en mayúsculas	"hola".upper()
lower()	Todo en minúsculas	"HOLA".lower()
title()	Primera letra de cada palabra en mayús.	"mi texto".title()
capitalize()	Primera letra mayús, resto mínus.	"hola".capitalize()
swapcase()	Invierte mayúsculas/minúsculas	"HoLa".swapcase()
isupper()	True si todo está en mayúsculas	"HOLA".isupper()
islower()	True si todo está en minúsculas	"hola".islower()
count()	Cuenta ocurrencias de un substring	"aabaa".count("a")
startswith()	True si empieza con el substring	"hola".startswith("ho")
endswith()	True si termina con el substring	"hola".endswith("la")
center()	Centra el texto con relleno	"hi".center(10, "*")

Método	Qué hace	Ejemplo
ljust()	Alinea a la izquierda con relleno	"hi".ljust(10)
rjust()	Alinea a la derecha con relleno	"hi".rjust(10, "0")

Método	Qué hace	Ejemplo
	Funciona de forma ascendente por defecto.	
reverse()	Invierte la lista en sitio	lista.reverse()
clear()	Vacía la lista (queda [])	lista.clear()
len()	Número o longitud de elementos	len(lista)

Regla de oro: `append([4,5])` agrega la lista como UN elemento. `extend([4,5])` agrega los elementos individualmente.

List Comprehension

Consiste en una construcción que permite crear listas a partir de otras listas.

Cada una de estas construcciones consta de una expresión que determina cómo modificar el elemento de la lista original, seguida de una o varias clausulas for y opcionalmente una o varias clausulas if.

```
# [expresion for variable in iterable]
# [expresion for variable in iterable if condicion]

numeros = [1, 2, 3, 4, 5]

# Doble de cada número
dobles = [n * 2 for n in numeros]
# [2, 4, 6, 8, 10]

# Solo los pares duplicados
dobles_pares = [n * 2 for n in numeros if n % 2 == 0]
# [4, 8]
```

Regla de oro: Si al leerla en voz alta no se entiende en 5 segundos, escribela como bucle. La comprension existe para claridad, no para impresionar.

Es un tipo de colección ordenada. pueden contener cualquier tipo de dato: números, cadenas, booleanos, ... y también listas.

Creación: Se realiza al indicar entre corchetes [], y separados por comas, los valores que se incluirán en la lista:

```
etapas = ["Física", "Antecedentes", "Entrevista"]

etapas[0]      # "Física"
etapas[-1]     # "Entrevista"
etapas[1:3]    # ["Antecedentes", "Entrevista"]
etapas[::-1]   # invertida
```

Métodos de Listas

Método	Qué hace	Ejemplo
append(x)	Agrega x al final	lista.append("nuevo")
extend(it)	Agrega elementos de un iterable al final	lista.extend([4, 5])
insert(i,x)	Inserta x en la posición i	lista.insert(1, "B")
pop(i)	Elimina y devuelve el elemento en i	lista.pop(2)
remove(x)	Elimina la primera ocurrencia de x	lista.remove("A")
index(x)	Devuelve la posición de la primera x	lista.index("A")
count(x)	Cuántas veces aparece x	lista.count("A")
sort()	Ordena la lista en sitio (modifica)	lista.sort()
sorted()	Crea nueva lista ordenada sin modificar el original.	Variable = sorted(lista)

BLOQUE 2

CONTROL DE FLUJO

Condicionales · Bucles · break/continue · Patrones

Sentencias Condicionales

Simple — if

Se ejecuta SOLO si la condición es verdadera.

```
if puntaje >= 60:
    print("Aprobado")
```

Compuesta — if / else

Rama verdadero Y rama falso. Solo se ejecuta UNA de las dos.

```
if puntaje >= 60:
    print("Aprobado")
else:
    print("Reprobado")
```

Múltiple — if / elif / else

Evalúa varias condiciones en orden. Ejecuta la primera que sea verdadera y descarta el resto.

```
if puntaje >= 90:
    nivel = "Excelente"
elif puntaje >= 70:
    nivel = "Bueno"           # ya sabemos puntaje < 90
elif puntaje >= 60:
    nivel = "Aceptable"      # ya sabemos puntaje < 70
else:
    nivel = "Insuficiente"
```

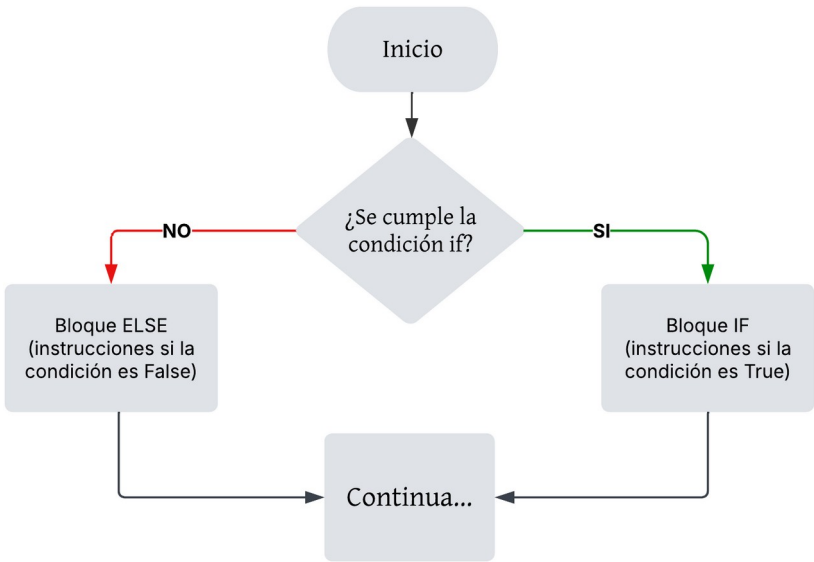
Anidadas — if dentro de if

```
if es_analista:
    if puntaje >= 80:
```

```
print("Analista aprobado")
else:
    print("Analista reprobado")
else:
    print("No es analista")
```

◆ **Patrón 01 — Condición encadenada** [IF / ELIF / ELSE]
Python evalúa los elif en orden y se detiene en el primero verdadero. No necesitas repetir la condición inferior. Útil para: notas, edades, rangos de precios.

Mapa de Proceso — if / else



Mapa de Proceso — if / elif / else

BLOQUE 3

ESTRUCTURAS DE DATOS

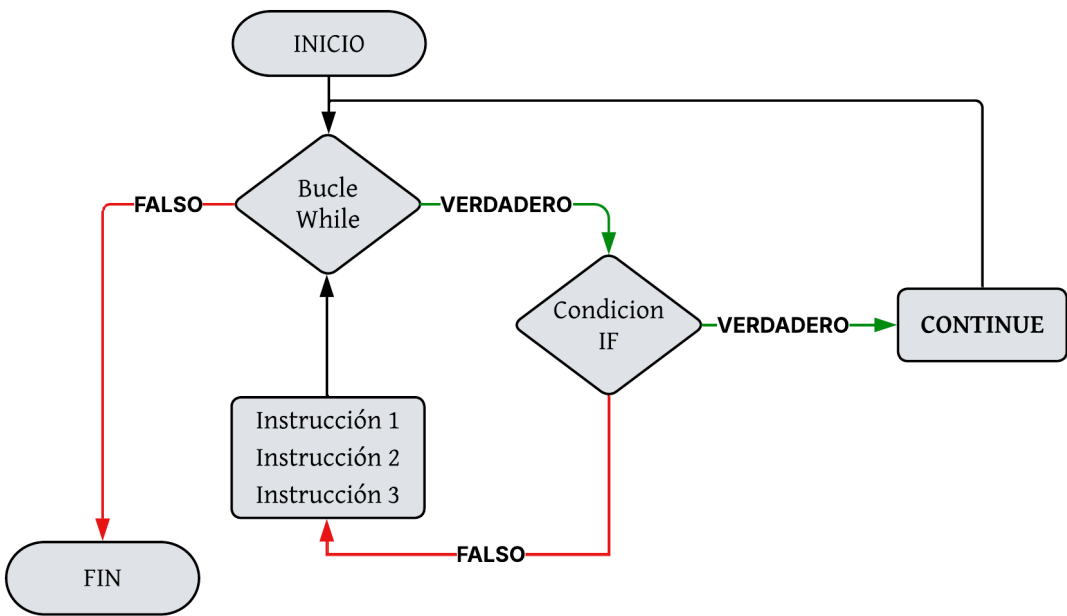
Listas · Diccionarios · Tuplas · Sets

¿Cuándo usar qué estructura?

Estructura	Ordenada	Mutable	Duplicados	Clave:Valor	Usar cuando...
Lista	Sí	Sí	Sí	No	Colección ordenada de elementos del mismo tipo
Tupla	Sí	No	Sí	No	Datos que NO deben cambiar (coordenadas, fechas)
Set	No	Sí	No	No	Elementos únicos, operaciones de conjunto
Diccionario	No*	Sí	claves No	Sí	Buscar por etiqueta: nombre, cargo, código

Regla de oro: Si tus datos tienen etiquetas naturales (nombre, edad, cargo), usa diccionario. Si son una secuencia sin etiquetas, usa lista. Si no deben cambiar, usa tupla.

Listas



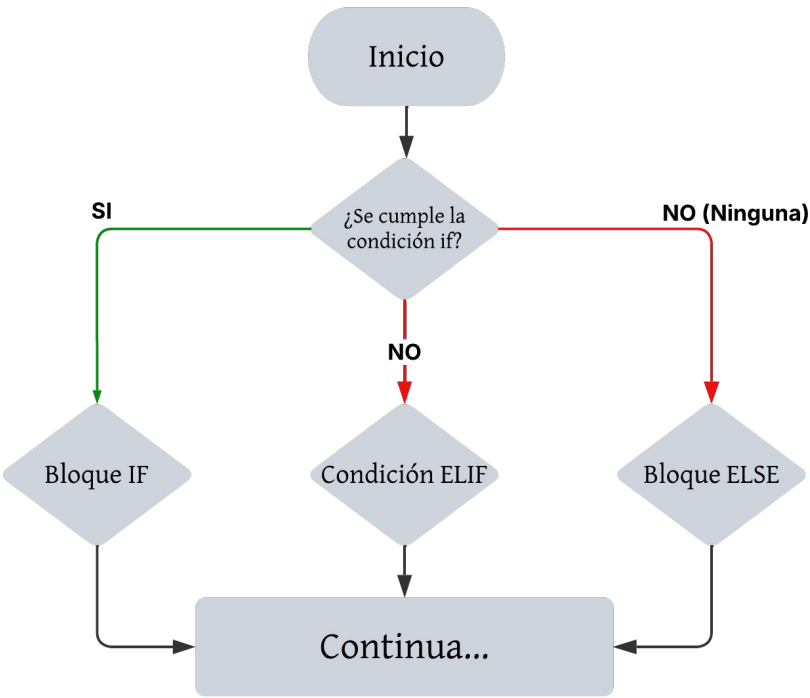
 Ejercicio: Script de planeación

Implementa el algoritmo de la sección Bloque 0 en código Python.

Entradas: cantidad_carpetas=5015, analistas=20, supervisores=10,
fecha_inicio=1, fecha_final=23, meta_a=12, meta_s=24

Salida: dos tablas formateadas con f-strings (., :.2% :< :>)

Tip: usa range(fecha_inicio, fecha_final) para el bucle de días.

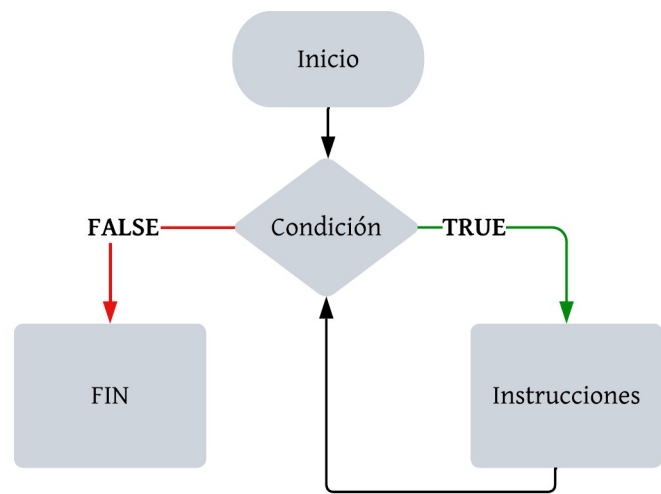


Bucle while

Repite un bloque de instrucciones MIENTRAS la condición sea True.

```
while condición:
    instruccion_1
    instruccion_2
    instruccion_3
    instruccion_4
```

Mapa de Proceso — Bucle while



```
# Ejemplo: validación de contraseña
clave_correcta = "12345"
intento = ""

while intento != clave_correcta:
    intento = input("Ingrese su clave: ")
    if intento != clave_correcta:
        print("Clave incorrecta. Intente de nuevo.")

print("Acceso concedido.")
```

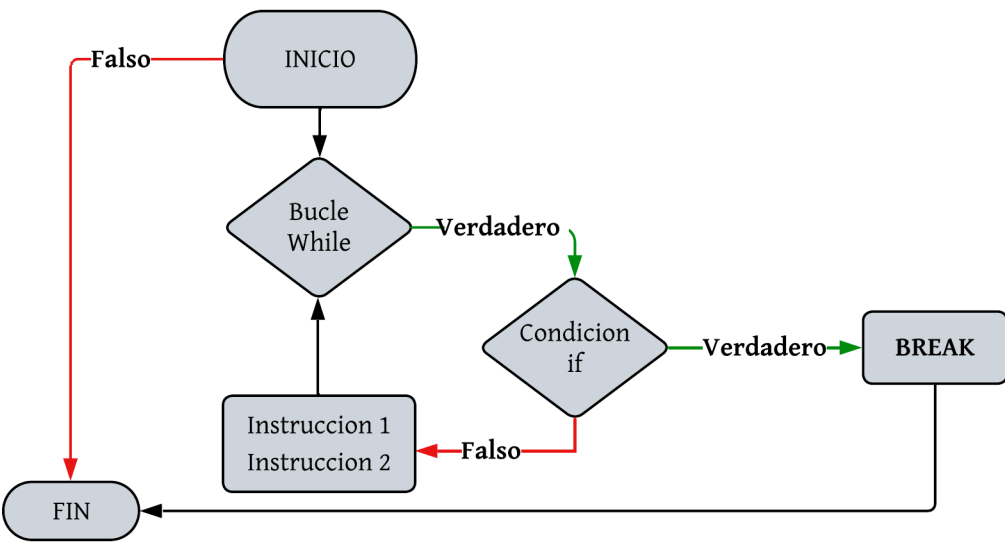
Regla de oro: Un bucle infinito ocurre cuando la condición de parada *NUNCA* se cumple. Siempre asegurate de que la variable que controla el while cambie dentro del bucle.

◆ Patrón 07 — Bucle infinito controlado [WHILE + IF + BREAK]

Repite indefinidamente hasta que se cumpla una condicion de exito. break es la única salida, va en el caso exitoso.

contador = 0

while True:
 entrada = input("...")



continue — saltar iteración
Salta el resto del bloque actual y pasa a la siguiente iteración.

```
for i in range(10):
    if i % 2 == 0:
        continue # salta los pares
    print(i)      # imprime 1, 3, 5, 7, 9
```

Mapa de Proceso — Sentencia continue

Guarda el ÍNDICE, no el valor — así puedes acceder a nombres[campeon] y cualquier lista paralela al final del bucle.

```
campeon = 0
peor     = 0

for i in range(len(lista)):
    if lista[i] > lista[campeon]:
        campeon = i
    if lista[i] < lista[peor]:
        peor = i

print(lista[campeon]) # el mayor
print(lista[peor])   # el menor
```

Sentencias break y continue

break — salir del bucle

Detiene la ejecución del bucle completamente y sale de él.

```
for i in range(10):
    if i == 5:
        break          # sale cuando i == 5

    print(i)           # imprime 0, 1, 2, 3, 4
```

Mapa de Proceso — Sentencia break

```
contador += 1

if condicion_A:
    print("pista A")
elif condicion_B:
    print("pista B")
else:
    print("Lograste!")
    break

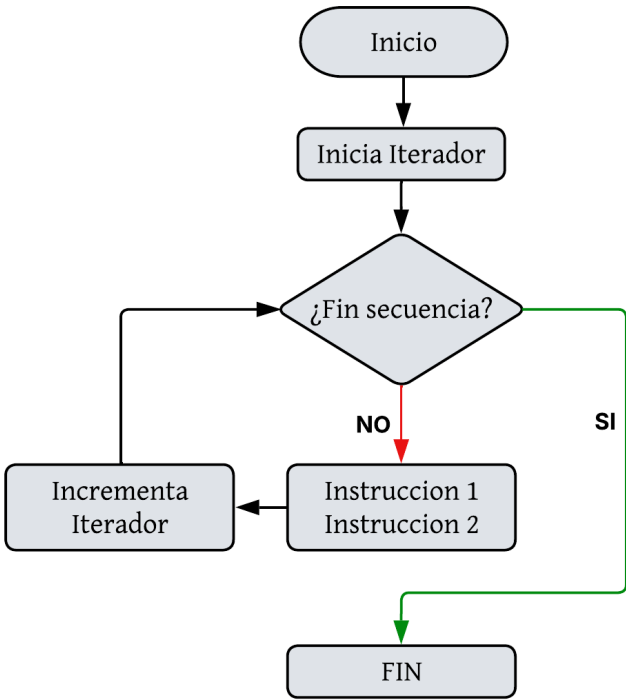
print(f"Intentos: {contador}")
```

Bucle for

Recorre un objeto iterable (lista, rango, cadena) ejecutando el bloque por cada elemento.

```
for variable in objeto_iterable:
    instruccion_1
    instruccion_2
```

Mapa de Proceso — Bucle for



```
# Ejemplo: acumular ventas
precios = [10.50, 20.00, 5.25, 100.00]
total = 0

for precio in precios:
    total += precio
    print(f"Producto: ${precio:.2f}")

print(f"Total: ${total:.2f}")
```

Regla de oro: for siempre debe tener un objeto iterable: cadena, lista, range(), enumerate(), zip(), etc.

range() — el aliado del for

Llamada	Genera	Uso típico
range(5)	[0,1,2,3,4]	Repetir N veces
range(1,6)	[1,2,3,4,5]	Del 1 al 5
range(0,10,2)	[0,2,4,6,8]	De 2 en 2
range(5,0,-1)	[5,4,3,2,1]	Cuenta regresiva

◆ **Patrón 02 — Bucle for con range()** *[FOR]*

range(1, 13) va del 1 al 12 — el último número NUNCA se incluye. range(n) genera 0..n-1.

```
for i in range(inicio, fin):

    # i vale inicio, inicio+1, ..., fin-1
    print(i)
```

◆ **Patrón 04 — Acumulador** *[FOR + SUMA]*

La variable empieza en 0 y crece en cada vuelta. Útil para total de ventas, suma de notas, valor del inventario.

```
total = 0

for precio in precios:
    total += precio

print(f"Total: ${total:,.0f}")
```

◆ **Patrón 05 — Contador condicional** *[FOR + IF]*

El contador solo sube cuando la condición es verdadera. Combina con acumulador para calcular promedio: total/contador.

```
contador = 0

for nota in notas:
    if nota < 60:
        contador += 1
print(f"Reprobados: {contador}")
```

◆ **Patrón 06 — Campeón vigente (máx/mín)** *[FOR + IF + ÍNDICE]*